

ЗАДАЧА Employee – модификатори за достъп

Условието е от г-жа В. Маринова:

Задача 1: Анализ на достъпност на членове при наследяване

Даден е базов клас **Employee**, който съдържа шест полета – по едно за всеки основен модификатор за достъп в C#: `public`, `private`, `protected`, `internal`, `protected internal`, `private protected`.

Твоите задачи са:

1. Да създадеш производен клас **Manager**, който се намира в **същия проект**, но в отделен namespace.
2. Да създадеш втори производен клас **ExternalManager**, който е в **друг проект**, но наследява **Employee**.
3. Да анализираш и опишеш в коментар към кода:
 - кои полета са достъпни в класа **Manager** и защо;
 - кои полета са достъпни в **ExternalManager** и защо;
 - кои полета не могат да бъдат достъпени и какъв модификатор причинява това.
4. Да демонстрираш в `Main()` чрез инстанции какви членове могат или не могат да се достъпват.

Цел: Да покажеш разбиране на достъпността при наследяване и зависимостта от проекта (assembly).

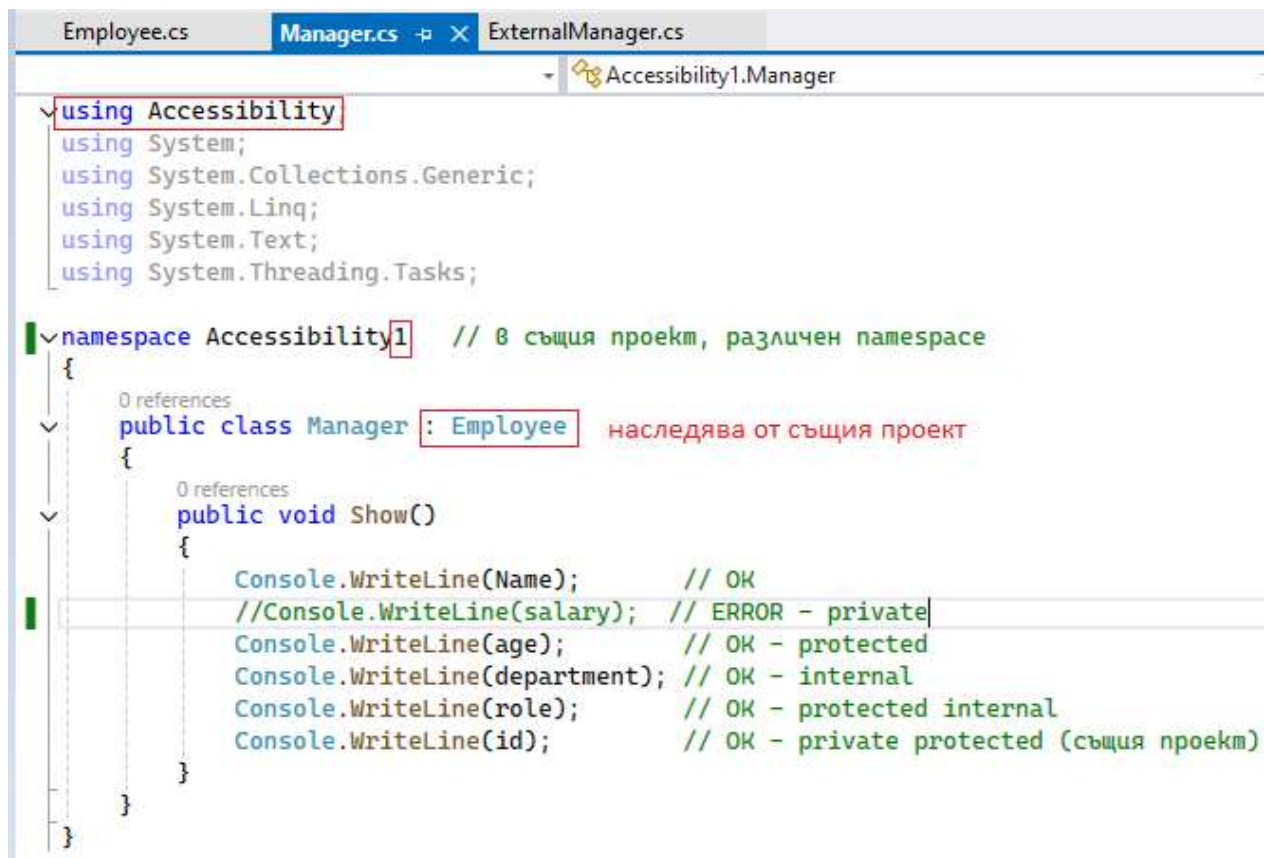
Решение: Създаваме нов проект Accessibility (той дава името на namespace). В отделен файл (Add -> new class) създаваме нов клас **Employee** и клас **Manager**, както се вижда на скрийншота:

```
Program.cs Employee.cs Manager.cs ExternalManager.cs
Accessibility Accessibility.Program

1 namespace Accessibility
2 {
3     0 references
4     internal class Program
5     {
6         0 references
7         static void Main(string[] args)
8         {
9             Console.WriteLine("Hello, World!");
10        }
11    }
```

```
Program.cs Employee.cs Manager.cs ExternalManager.cs
Accessibility Accessibility.Employee

6 namespace Accessibility
7 {
8     2 references
9     public class Employee
10    {
11        public string Name; // достъпен навсякъде
12        private int salary; // достъпен само в Employee
13        protected int age; // достъпен в наследници
14        internal string department; // достъпен само в същия проект
15        protected internal string role; // достъпен в наследници или в същия проект
16        private protected int id; // достъпен в наследници в същия проект
17    }
```

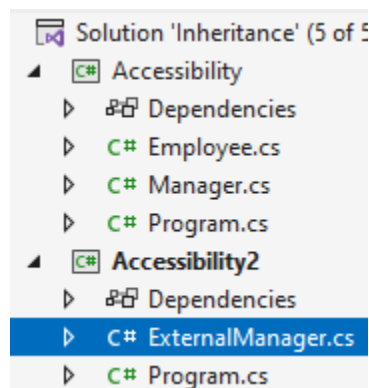


```
Employee.cs  Manager.cs  ExternalManager.cs
Accessibility1.Manager

using Accessibility1
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Accessibility1 // в същия проект, различен namespace
{
    0 references
    public class Manager : Employee // наследява от същия проект
    {
        0 references
        public void Show()
        {
            Console.WriteLine(Name); // OK
            //Console.WriteLine(salary); // ERROR - private
            Console.WriteLine(age); // OK - protected
            Console.WriteLine(department); // OK - internal
            Console.WriteLine(role); // OK - protected internal
            Console.WriteLine(id); // OK - private protected (същия проект)
        }
    }
}
```

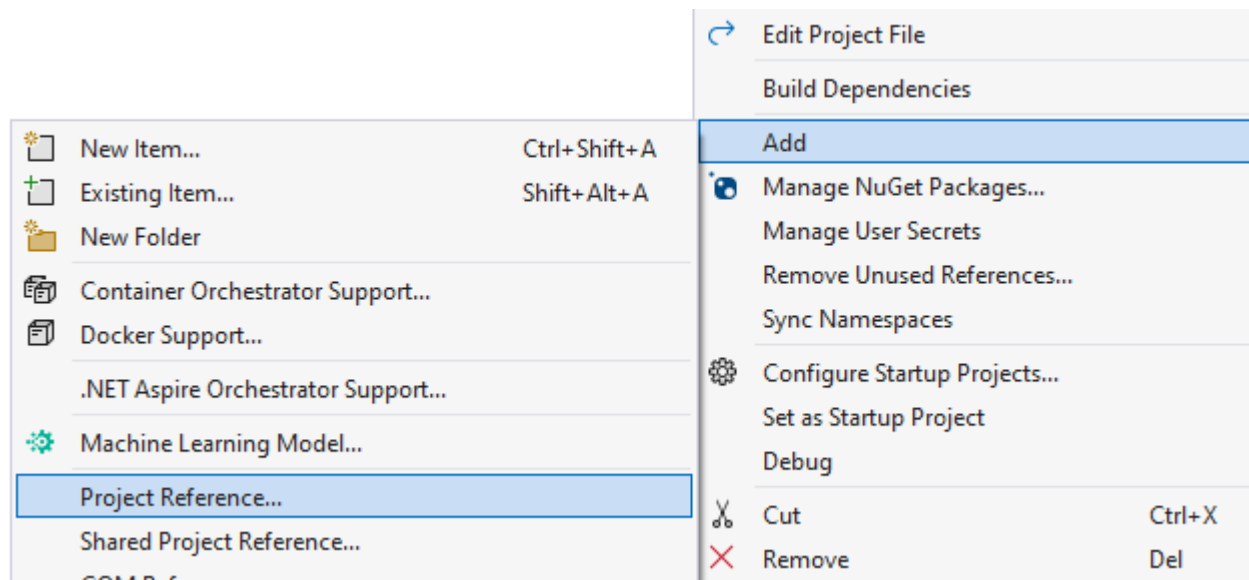
Добавяме нов проект в рамките на същия Solution:

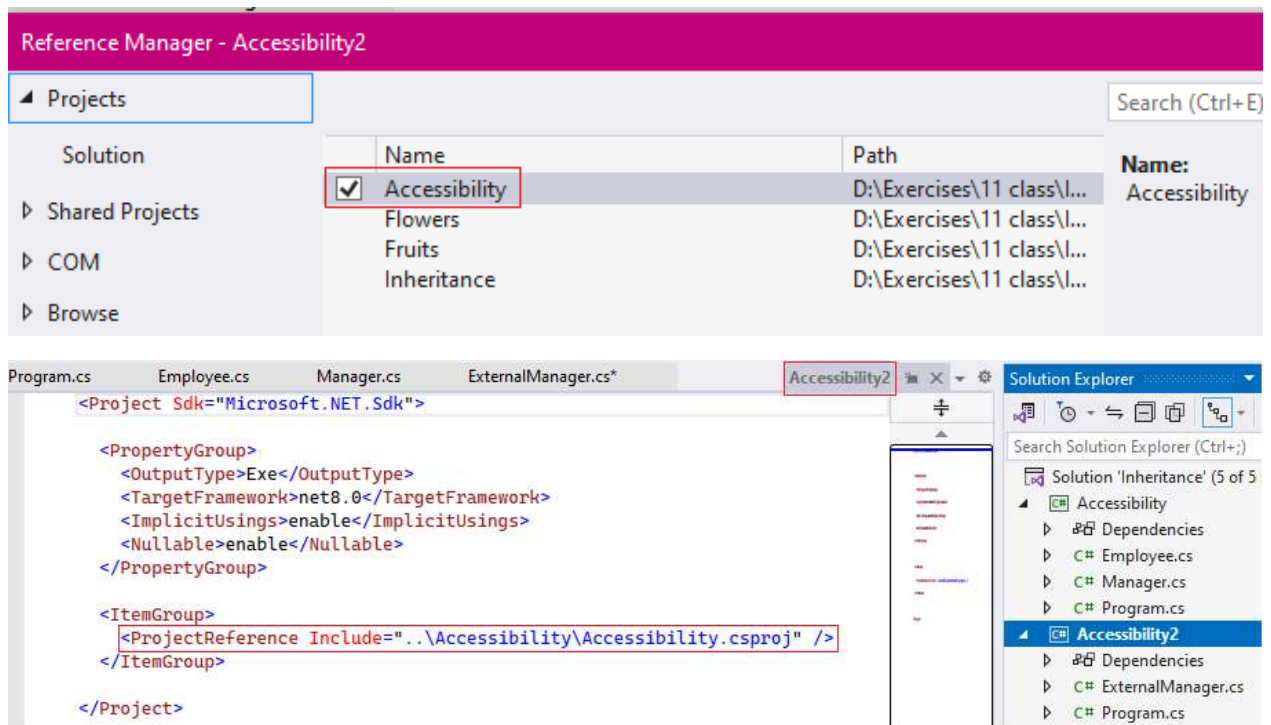


```
Employee.cs  Manager.cs  ExternalManager.cs*  Accessibility2.ExternalManager  SI
using Accessibility;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Accessibility2
{
    0 references
    public class ExternalManager : Employee
    {
        0 references
        public void Show()
        {
            Console.WriteLine(Name);           // OK - public
            //Console.WriteLine(salary);        // ERROR - private
            Console.WriteLine(age);             // OK - protected
            //Console.WriteLine(department);    // ERROR - internal
            Console.WriteLine(role);            // OK - protected internal
            //Console.WriteLine(id);            // ERROR - private protected (гръз npoekm)
        }
    }
}
```

Забележка: Възможно е някои IDE да изискват допълнителна настройка на втория проект за референция към родителя, за да бъде разпознат:





Кодът на Employee.cs:

```
namespace Accessibility
{
    public class Employee
    {
        public string Name;           // достъпен навсякъде
        private int salary;           // достъпен само в Employee
        protected int age;            // достъпен в наследници
        internal string department;    // достъпен само в същия проект
        protected internal string role; // достъпен в наследници или в същия проект
    }
}
```

Кодът на Manager.cs:

```
using Accessibility;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Accessibility1 // в същия проект, различен namespace
{
    public class Manager : Employee
    {
        public void Show()
        {
            Console.WriteLine(Name); // OK
        }
    }
}
```

```

        //Console.WriteLine(salary); // ERROR - private
        Console.WriteLine(age);      // OK - protected
        Console.WriteLine(department); // OK - internal
        Console.WriteLine(role);     // OK - protected internal
        Console.WriteLine(id);       // OK - private protected (същия проект)
    }
}

```

Кодът на ExternalManager.cs:

```

using Accessibility;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Accessibility2
{
    public class ExternalManager : Employee
    {
        public void Show()
        {
            Console.WriteLine(Name); // OK - public
            //Console.WriteLine(salary); // ERROR - private
            Console.WriteLine(age); // OK - protected
            //Console.WriteLine(department); // ERROR - internal
            Console.WriteLine(role); // OK - protected internal
            //Console.WriteLine(id); // ERROR - private protected (друг
проект)
        }
    }
}

```

Референция в проекта, който наследява класа:

```

<ItemGroup>
  <ProjectReference Include="..\Accessibility\Accessibility.csproj" />
</ItemGroup>

```